

Activity: OOP Concepts Using Real-World Scenarios

CSA0910 JAVA

Sameer Ahmed G

192421154

Q1. Single Inheritance – Scenario

Scenario

A company has a Vehicle class with methods startEngine() and stopEngine(). A Car class extends Vehicle and adds a playMusic() method. The company now wants every car to automatically log the engine start time whenever startEngine() is called while still using the original engine-start functionality.

Answers

1. How would you implement this using single inheritance?

Create a Car class that extends the Vehicle class. The Car class inherits all the methods of Vehicle and adds its own method playMusic(). It overrides startEngine() to include logging functionality.

2. Should Car override startEngine()? Why?

Yes. Overriding allows the Car class to add logging functionality while retaining the original engine-start behavior.

3. How can the parent class implementation still be executed?

The parent class implementation can be executed using the super.startEngine() statement inside the overridden method.

4. What happens if Car does not override startEngine()?

The Car object will directly use the inherited startEngine() method from the Vehicle class without any logging functionality.

Program

```
class Vehicle {  
    void startEngine() {  
        System.out.println("Engine Started");  
    }  
  
    void stopEngine() {  
        System.out.println("Engine Stopped");  
    }  
}
```

```
}  
}
```

```
class Car extends Vehicle {
```

```
    @Override
```

```
    void startEngine() {
```

```
        super.startEngine();
```

```
        System.out.println("Engine start time logged.");
```

```
    }
```

```
    void playMusic() {
```

```
        System.out.println("Playing Music");
```

```
    }
```

```
}
```

```
public class SingleInheritanceDemo {
```

```
    public static void main(String[] args) {
```

```
        Car car = new Car();
```

```
        car.startEngine();
```

```
        car.playMusic();
```

```
        car.stopEngine();
```

```
    }
```

```
}
```

Q2. Multilevel Inheritance – Scenario

Scenario

A software company models employees as:

- Employee → stores employee ID and name.
- Developer → stores programming language.
- SeniorDeveloper → stores team size and performs code reviews.

HR wants every SeniorDeveloper to access employee information and programming language details without duplicating code.

Answers

1. Why is multilevel inheritance suitable here?

Multilevel inheritance allows each derived class to inherit all properties and methods from its parent class, avoiding code duplication.

2. Which members are accessible in SeniorDeveloper?

SeniorDeveloper can access employee ID, employee name, programming language, team size, and any inherited methods.

3. If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?

SeniorDeveloper inherits the overridden version present in the Developer class.

4. Draw the inheritance hierarchy.

Employee

|

Developer

|

SeniorDeveloper

Program

```
class Employee {  
    int empId = 101;  
    String name = "Sameer";  
  
    void displayEmployee() {  
        System.out.println(empId + " " + name);  
    }  
}
```

```
}
```

```
class Developer extends Employee {
```

```
    String language = "Java";
```

```
    void displayLanguage() {
```

```
        System.out.println("Language: " + language);
```

```
    }
```

```
}
```

```
class SeniorDeveloper extends Developer {
```

```
    int teamSize = 5;
```

```
    void displayTeam() {
```

```
        System.out.println("Team Size: " + teamSize);
```

```
    }
```

```
}
```

```
public class MultilevelInheritanceDemo {
```

```
    public static void main(String[] args) {
```

```
        SeniorDeveloper sd = new SeniorDeveloper();
```

```
        sd.displayEmployee();
```

```
        sd.displayLanguage();
```

```
        sd.displayTeam();
```

```
    }
```

```
}
```

Q3. Hierarchical Inheritance – Scenario

Scenario

An online shopping application has a base class Payment. Three classes inherit from it:

- CreditCardPayment
- UPIPayment
- NetBankingPayment

Each payment type processes transactions differently but shares common validation logic.

Answers

1. Why is hierarchical inheritance appropriate here?

Hierarchical inheritance is suitable because multiple payment methods share common functionality while implementing different transaction processing methods.

2. Where should common validation logic be implemented?

The validation logic should be implemented in the parent Payment class.

3. How can each payment class provide its own transaction processing?

Each subclass should define or override its own processPayment() method.

4. What is the advantage over writing three independent classes?

It reduces code duplication, improves maintainability, and promotes code reusability.

Program

```
class Payment {  
  
    void validate() {  
        System.out.println("Payment Validated");  
    }  
}  
  
class CreditCardPayment extends Payment {  
  
    void processPayment() {  
        System.out.println("Processing Credit Card Payment");  
    }  
}  
class UPIPayment extends Payment {
```

```
void processPayment() {  
    System.out.println("Processing UPI Payment");  
}  
}  
  
class NetBankingPayment extends Payment {  
  
    void processPayment() {  
        System.out.println("Processing Net Banking Payment");  
    }  
}  
  
public class HierarchicalInheritanceDemo {  
  
    public static void main(String[] args) {  
  
        CreditCardPayment c = new CreditCardPayment();  
        c.validate();  
        c.processPayment();  
  
        UPIPayment u = new UPIPayment();  
        u.validate();  
        u.processPayment();  
  
        NetBankingPayment n = new NetBankingPayment();  
        n.validate();  
        n.processPayment();  
    }  
}
```

Q4. Compile-Time Polymorphism (Method Overloading)

Scenario

A calculator supports addition of two integers, three integers, and two decimal numbers. The user should call the same method name add().

Answers

1. Which OOP concept should be used?

Compile-time polymorphism (Method Overloading).

2. How will Java decide which method to invoke?

Java selects the method based on the number, type, and order of the arguments during compilation.

3. Can methods be overloaded by changing only the return type?

No. Methods cannot be overloaded only by changing the return type.

4. What happens if add(5,10.0) is called?

Java performs automatic type conversion and invokes the add(double,double) method.

Program

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {
```

```

Calculator c = new Calculator();

System.out.println(c.add(5,10));
System.out.println(c.add(5,10,15));
System.out.println(c.add(5.5,4.5));
System.out.println(c.add(5,10.0));
}
}

```

Q5. Runtime Polymorphism (Method Overriding)

Scenario

Different restaurants calculate delivery time differently, but the application stores all restaurant objects using the parent class reference.

Answers

1. Which OOP concept is being used?

Runtime Polymorphism (Method Overriding).

2. Which version of calculateDeliveryTime() executes?

The overridden method of the actual object executes.

3. When is the method selection decided?

The method selection is decided at runtime.

4. What would happen if calculateDeliveryTime() were declared static?

Static methods are not overridden. The parent class method would be called based on the reference type.

Program

```

class Restaurant {

    void calculateDeliveryTime() {

        System.out.println("General Delivery Time");

    }

}

```

```
class Pizza extends Restaurant {  
  
    void calculateDeliveryTime() {  
        System.out.println("30 Minutes");  
    }  
}
```

```
class Burger extends Restaurant {  
  
    void calculateDeliveryTime() {  
        System.out.println("20 Minutes");  
    }  
}
```

```
class Biryani extends Restaurant {  
  
    void calculateDeliveryTime() {  
        System.out.println("40 Minutes");  
    }  
}
```

```
public class RuntimePolymorphismDemo {  
  
    public static void main(String[] args) {  
  
        Restaurant r;  
  
        r = new Pizza();  
        r.calculateDeliveryTime();  
    }  
}
```

```

    r = new Burger();
    r.calculateDeliveryTime();

    r = new Biryani();
    r.calculateDeliveryTime();
}
}

```

Q6. Interface

Scenario

A smart home application supports Smart Bulb, Smart Fan, and Smart AC. Third-party manufacturers should also be able to integrate their own devices.

Answers

1. Would you use an interface or inheritance? Why?

An interface is more suitable because different manufacturers can implement the same functionality without inheriting a common implementation.

2. Design an appropriate interface.

```

interface SmartDevice {
    void turnOn();
    void turnOff();
}

```

3. Can one device implement multiple interfaces? Give an example.

Yes.

```

class SmartAC implements SmartDevice, WifiEnabled {
}

```

4. Why is an interface better than creating a common base class?

Interfaces provide flexibility, support multiple inheritance of type, and allow new device manufacturers to integrate their products without modifying existing classes.

Program

```

interface SmartDevice {

```

```
void turnOn();
```

```
void turnOff();
```

```
}
```

```
class SmartBulb implements SmartDevice {
```

```
    public void turnOn() {
```

```
        System.out.println("Bulb ON");
```

```
    }
```

```
    public void turnOff() {
```

```
        System.out.println("Bulb OFF");
```

```
    }
```

```
}
```

```
class SmartFan implements SmartDevice {
```

```
    public void turnOn() {
```

```
        System.out.println("Fan ON");
```

```
    }
```

```
    public void turnOff() {
```

```
        System.out.println("Fan OFF");
```

```
    }
```

```
}
```

```
public class InterfaceDemo {
```

```
public static void main(String[] args) {  
  
    SmartDevice bulb = new SmartBulb();  
    SmartDevice fan = new SmartFan();  
  
    bulb.turnOn();  
    fan.turnOn();  
  
    bulb.turnOff();  
    fan.turnOff();  
}  
}
```